UNIVERSIDADE DO ALGARVE
FACULDADE DE CIÊNCIAS E TECNOLOGIA

DESENVOLVIMENTO DE APLICAÇÕES PARA A WEB

Relatório do Projeto: "UAlly – University Ally"



Grupo 8:
Helena Vassal nº 75525
Rui Saraiva nº 79890
Vasco Evaristo nº 79806

Docentes:
Prof. Noélia Susana Costa Correia
Prof. Rúben David de Sousa Gomes

1. Introdução

O projeto "UAlly – University Ally" foi desenvolvido como parte da unidade curricular de Desenvolvimento de Aplicações Web do curso de Engenharia Informática da Universidade do Algarve. O objetivo do projeto é criar uma aplicação web para ajudar estudantes universitários a organizar suas tarefas, listas e categorias de forma eficiente.

2. Manual de uso

Configuração do Ambiente:

- **Instalar Node.js e npm:**
  - Navegar até as pastas “backend” e “frontend” e executar “npm install” para instalar as dependências.
- **Execução do Projeto:**
  - No backend, executar “npm run dev” para iniciar o servidor e esperar pela mensagem “Running...”.
  - No frontend, executar “npm start” para iniciar a aplicação React.

3. Layout

- **HelpView**

  Apresenta informações úteis sobre como utilizar o website. Explica como criar disciplinas e tarefas e como editar os dados da conta. É exibida através de um pop-up controlado por uma propriedade show, e pode ser fechado ao clicar no botão "Fechar".

- **AboutView**

  Fornece informações sobre os autores do projeto e o objetivo do website. O utilizador encontra um botão "Voltar" no canto superior esquerdo, que o redireciona para a WelcomeView.

- **LoggedMainView**

  É a página principal para os utilizadores autenticados. Após o login, o utilizador pode criar ou aceder às disciplinas e projetos que criou, criar novos trabalhos e tarefas.

- **LoginView**

  Apresenta um formulário com campos para o utilizador inserir o seu e-mail e

palavra-passe. Inclui um botão "Entrar", que submete os dados para autenticação, e outro botão "Registar", que redireciona o utilizador para a RegisterView caso ainda não tenha uma conta.
Além disso, esta view exibe mensagens de erro, como "Endereço de e-mail não registado" ou "Palavra-Passe incorreta", caso o login falhe. Também possui um botão para redirecionar para a AboutView, permitindo ao utilizador saber mais sobre o projeto.

- **ProfileView**

  Apresenta as informações do perfil do utilizador, como nome e e-mail, e permite editar os respetivos dados ou sair da conta. Esta view dá ao utilizador um controlo adicional sobre as suas preferências e dados pessoais.

- **RegisterView**

  Inclui um formulário onde o utilizador pode criar uma nova conta. Os campos deste formulário incluem informações como nome, e-mail e palavra-passe. Após o registo, o utilizador é redirecionado para a página de login para que possa aceder ao website.

- **ProjectView**

  Aqui poderão ser criadas ou visualizadas as tarefas referente ao trabalho selecionado.

- **WelcomeView**

  É a página inicial para utilizadores não autenticados. Nela, o utilizador encontra botões para se registar, caso ainda não tenha uma conta, fazer login ou aceder à AboutView para saber mais sobre o projeto.

4. Estrutura do Projeto

O projeto está organizado em duas partes principais: backend e frontend.

- **Backend**

  A pasta “backend” contém o código do servidor, incluindo a interação com a base de dados SQLite.

  - **serverInfo.json**: Contém informações de configuração do servidor, como detalhes de SMTP.

- **main.ts**: Arquivo principal que configura e inicia o servidor Express.
- **serverInfo.ts**: Lê e exporta as informações de configuração do servidor a partir de serverInfo.json.
- **Tabelas SQL**: Arquivos SQL que definem a estrutura das tabelas da base de dados.
  - CATEGORIES.sql
  - LISTS.sql
  - TASKS.sql
  - USERS.sql
- **Tabelas TypeScript**: Arquivos TypeScript que implementam a lógica de interação com as tabelas da base de dados.
  - Categories.ts
  - Lists.ts
  - Tasks.ts
  - Users.ts
- **tsconfig.json**: Arquivo de configuração do TypeScript.

Frontend

A pasta "frontend" contém o código da interface do utilizador, desenvolvido com React.

- **src**: Contém o código-fonte da aplicação React.
  - **App.tsx**: Componente principal da aplicação React.
  - **config.ts**: Contém a configuração do endereço do servidor.
  - **index.tsx**: Ponto de entrada da aplicação React.
  - **assets/styles**: Contém arquivos de estilo CSS.
    - App.css
    - BaseLayout.css
    - index.css
  - **components**: Contém componentes React reutilizáveis.
    - BaseLayout.tsx
    - Header.tsx
    - List.tsx
    - ModalPop.tsx
    - SideBar.tsx
    - Task.tsx
  - **views**: Contém as diferentes vistas da aplicação.
    - AboutView.tsx

        - HelpView.tsx
        - LoggedMainView.tsx
        - LoginView.tsx
        - ProfileView.tsx
        - RegisterView.tsx
        - WelcomeView.tsx
    - **state.ts**: Contém a lógica da getão de estado da aplicação.
    - **Categories.ts**: Contém a lógica de interação com as categorias.
    - **List.ts**: Contém a lógica de interação com as listas.
    - **Task.ts**: Contém a lógica de interação com as tarefas.
    - **Users.ts**: Contém a lógica de interação com os usuários.

- **tsconfig.json**: Arquivo de configuração do TypeScript.

5. Funcionalidades

- **Backend**:
    - **Gestão de Utilizadores**: Permite o registo, login, atualização e exclusão de utilizadores. As operações são definidas no arquivo Users.ts.
    - **Gestão de Tarefas**: Permite criar, atualizar, excluir e obter tarefas. As operações são definidas no arquivo Tasks.ts.
    - **Gestão de Categorias**: Permite criar, atualizar, excluir e obter categorias. As operações são definidas no arquivo Categories.ts.
    - **Gestão das listas**: Permite criar, atualizar, excluir e obter listas. As operações são definidas no arquivo Lists.ts.

- **Frontend**:
    - **Interface de Utilizador**: desenvolvida com React e Bootstrap, garantindo uma experiência mais fácil de usar e adaptável.
    - **Integração com o Backend**: O frontend comunica-se com o backend usando a biblioteca Axios para realizar operações CRUD.

6. 3 Extra Changes

- **Autenticação com encriptação de passwords**

  Implementámos um sistema de autenticação que utiliza encriptação para proteger as passwords dos utilizadores, garantindo que as passwords armazenadas na base de estará encriptada.
  Durante o registo ou login, a password inserida é comparada com a

armazenada de maneira a validar a identidade do utilizador.

- **Importar foto de perfil**
  Adicionámos a funcionalidade para que os utilizadores possam personalizar o seu perfil ao importar uma foto. A imagem é gerida no frontend utilizando o estado do React, armazenando-a.

- **Gestão de base de dados com SQLite**

É a página inicial para utilizadores não autenticados. Nela, o utilizador encontra botões para se registar, caso ainda não tenha uma conta, fazer login ou aceder à AboutView para saber mais sobre o projeto.

7. Tecnologias e Dependências Utilizadas

Backend: Node.js, Express, SQLite, TypeScript.
Frontend: React, Bootstrap, Axios, TypeScript.

Para desenvolvermos o projeto utilizámos as seguintes packages:
• Express – npm install express
• sqlite3 – npm install sqlite3

• Iniciar o projeto - npx create-react-app my-app
• Typescript - npm install --save typescript @types/node @types/react @types/react-dom @types/jest
• Bootstrap - npm install react-bootstrap bootstrap
• Mdb-react-ui-kit - npm i mdb-react-ui-kit
• Axios - npm i axios

8. Conclusão

O projeto "UAlly" alcançou o principal objetivo: uma aplicação web funcional e intuitiva para ajudar os utilizadores a organizar as suas vidas no contexto académico e pessoal. Durante o desenvolvimento, enfrentámos diversos desafios, como a integração entre o frontend e o backend, a implementação de funcionalidades de listas e tarefas, e a gestão eficiente de estados no React. Apesar dessas dificuldades, conseguimos superá-las, permitindo-nos aprofundar conhecimento. Contudo, nem todos os objetivos inicialmente propostos foram atingidos: a criação de lembretes, o registo de tempo e a partilha.